



Deprecate implicit conversion from `bool` to character types

Document number:

[P3765R0](#)

Date:

2025-07-12

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Co-authors:

Hana Dusíková <hanicka@hanicka.net>

GitHub Issue:

wg21.link/P3765/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/deprecate-bool-conv.cow



ABSTRACT: Implicit conversion from `bool` to character types serve little purpose in the language, and invite bugs. We propose to deprecate them.

§ Contents

- 1 Introduction
- 2 Proposal
 - 2.1 Explicit casts should be allowed
- 3 Impact on existing code
- 4 Implementation experience
- 5 Wording
- 6 References

§ 1. Introduction



BUG: Consider the following bug, resulting from a typo (parentheses in the wrong place):

```
std::string_view str = /* ... */;
if (str.ends_with('\n' || str.ends_with('\r'))) {
    // ...
}
```

`'\n' || str.ends_with('\r')` is equivalent to `true`, and the `ends_with(char)` overload is called with `char(true)`.

Surprisingly, every compiler accepts this code at the time of writing, with no warning and no errors. That is because `bool` $\rightarrow$ `char` conversion is neither a narrowing conversion, nor does it change the sign of the operand, so it does not trigger any checks.

Semantically (assuming ASCII-compatible encoding), in `bool` $\rightarrow$ `char` conversion, `false` is converted to U+0000 NULL, and `true` is converted to U+0001 START OF HEADING. It is hard to see how this conversion needs to be a built-in language feature; possibly, no C++ developer ever wanted a conversion to have these semantics.

Seeing that these implicit conversions also invite bugs like the one above, that makes them not just useless, but actively harmful to the language, qualifying them for removal. Similarly, [P1120R0] deprecated array-to-pointer conversion in equality comparison of two arrays, a bug-prone and virtually useless conversion.

§ 2. Proposal

The goal of this proposal is to deprecate implicit conversion from `bool` to character types. That is, to `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t`.

Conversions to `signed char` and `unsigned char` are not affected. That is because these types are used for numerics and not to represent characters, so e.g. `bool` $\rightarrow$ `unsigned char` represents a conversion from `false` to `0` and from `true` to `1`, which is sometimes used in branchless programming:



EXAMPLE: In the following example, `z` is incremented if `x` is odd.

```
int x = /* ... */, z;
unsigned char y = x % 2 != 0;
z += y;
```

Note that branchless programming is generally not affected by a `bool` $\rightarrow$ `char` deprecation. For example, in the expression `'0' + (x % 2 != 0)`, both operands are converted to `int` due to the usual arithmetic conversions and integral promotion. `bool` $\rightarrow$ `char` conversion actually

takes place when calling a function with `char` parameter with a `bool` argument, when storing a `bool` value in a `char` variable, and other such "direct" scenarios.



§ 2.1. Explicit casts should be allowed

While an implicit `bool` $\rightarrow$ `char` conversion is likely a bug, we want users to have a way to suppress deprecation warnings and to retain the old behavior, in the unlikely event they wanted the conversion to happen. There are various other conversion, like `std::nullptr_t` $\rightarrow$ `bool` or `void*` $\rightarrow$ `int*` which follow this pattern of "explicit conversions only".

In summary, the intended semantics are as follows:

```
bool b = /* ... */;

char d1 = b;           // deprecated
char d2(b);           // deprecated
char d3{b};           // deprecated
char d4 = {b};         // deprecated
char d5 = char{b};     // deprecated

char k1 = char(b);     // OK
char k2 = (char)b;     // OK
char k3 = static_cast<char>(b); // OK

'a' + b;              // OK
```

While most of these are obvious, some decisions require rationale:

- d2 should be deprecated because direct-initialization, unlike casts, does not generally opt into all possible conversions. For example, `static_cast<int*>(void_ptr)` is valid, while `int* p(void_ptr)`, for some `void* void_ptr`.
- d5 should be deprecated because list-initialization is generally more restrictive. For instance, it does not permit narrowing conversions, and `I{void_ptr}` is ill-formed while `I(void_ptr)` is okay, where `I` is `int*`.
- k1 and k2 should be okay because they are defined in terms of `static_cast`.

§ 3. Impact on existing code

During discussion of this proposal with various members of the C++ community, no one was able to produce an example of where this conversion is used in existing code. However, we did not yet perform an extensive survey of large code bases.

Furthermore, there are GCC targets that do not comply with the C99 standard, where `int8_t` is defined to be `char` (not `signed char`). The following snippet includes GCC definitions for Solaris 2 ([GccSolaris2]):



```
/* ??? This definition of int8_t follows the system header but does
   not conform to C99. Likewise int_fast8_t, int_least8_t. */
#define INT8_TYPE "char"
```



Since `bool` → `int8_t` conversion could plausibly happen in branchless programming, hypothetically, some code on such non-compliant platforms is affected.

In any case, deprecation warnings are easily resolved by performing an explicit conversion, or by prepending `+` to the `bool` operand, which would trigger integral promotion and result in `int` → `char` conversion instead.

§ 4. Implementation experience

A [\[ClangWarning\]](#) has been implemented in a fork, which triggers in the deprecated cases.

§ 5. Wording

The wording changes are relative to [\[N5008\]](#).

Change [\[conv.integral\]](#) paragraph 1 as follows:



A prvalue of an integer type [S](#) can be converted to a prvalue of another integer type [D](#). The conversion is deprecated ([\[depr.conv.bool.char\]](#)) if

- [S is bool](#) and [D is a character type \(\[\\[basic.fundamental\\]\]\(#\)\)](#) and
- the conversion is not necessitated by a `static_cast` ([\[expr.static.cast\]](#)).

[Note: This deprecation also applies to cv-qualified character types because prvalues of such types are adjusted to cv-unqualified types ([\[expr.type\]](#)). — end note]

A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.

Insert a new paragraph immediately following [\[conv.integral\]](#) paragraph 1:



A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.

Insert a new subclass in [\[depr\]](#) between [\[depr.local\]](#) and [\[depr.capture.this\]](#), containing a single paragraph:



Conversion from `bool` to a character type

Integral conversion ([\[conv.integral\]](#)) from a prvalue of [\[depr.conv.bool.char\]](#) type `bool` to a prvalue of character type ([\[basic.fundamental\]](#)) is deprecated, unless it

was necessitated by a `static_cast`.

[Example:

```
char c = true;           // deprecated
char{true};             // deprecated

static_cast<char>(true); // OK, conversion directly necessitated by a static_cast
char(true);             // OK, conversion indirectly necessitated by a static_cast
c + true;               // OK, c and true are promoted to int

— end example]
```



§ 6. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P1120R0] Richard Smith. Consistency improvements for `<=>` and other comparison operators (2018-06-08)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1120r0.html>

[GccSolaris2] Operating system specific defines to be used when targeting GCC for any Solaris 2 system
<https://gcc.gnu.org/cgit/gcc/tree/gcc/config/sol2.h#n48>

[ClangWarning] Hana Dusíková. Implementation of Clang warning for this proposal
<https://github.com/llvm/llvm-project/commit/d7b80255bd582ac93d83f1ac5401b7787fee66c7>